



S. G. Ganesh

Liskov's Substitution Principle

LSP is a cardinal rule to follow in object-oriented designs. In this column, we'll introduce LSP to those new to OOP (Object Oriented Programming), and discuss a couple of examples from JDK that violate this principle.

The whole of object-oriented design boils down to a few principles such as abstraction, encapsulation, modularisation, hierarchy, and regularity.

Abstraction allows us to model in terms of commonality between objects, and express the design in terms of the problem domain. Encapsulation allows bundling data and the functions operating on it as a single unit; further encapsulation enables us to hide implementation details, and expose only the interface to the users. Modularisation allows separation of concerns and definition of crisp boundaries between abstractions. With hierarchy, we can organise or arrange abstractions at multiple levels. With regularity, we can create uniform solutions that are easier to grasp and understand. It takes years of experience to understand that these are the essence of OOP (and not the favourite features that your programming language provides), and following these generic principles gives us incredible power in problem solving.

Let us take the hierarchy principle, for example. One way to realise hierarchy is to create a relationship between abstractions, using a language feature named inheritance. For example, in an image-processing application, instead of writing code in terms of different kinds of image files such as GIF, JPEG, EPS, SVG, PNS, etc, one can exploit the commonalities between the image file formats. For example, we can classify image types at a higher level as raster (storing images as bitmaps, or in terms of pixels), or vector (geometric description of images) formats. This allows for abstracting commonalities of the specific file format, and moving the specific details of a file format to that class. In other words, we can have a base class named *ImageFile*, and two derived classes, namely, *RasterFile* and *VectorFile*. Further, *RasterFormat* can have derived classes such as *JPEGFile*, *TIFFFile*, *PNGFile*, etc, and *VectorFile* can have derived classes such as *CGMFile*, *SVGFile*, etc. With this design, when we want to write high-level code—

for example, reading the file from the disk—we can write code in terms of the generic type *ImageFile*. Now, if there is any need to refer to specific types—for example, when converting from one image type to another—we can use specific file types such as *JPEGFile*, *SVGFile* etc. With this design approach, it is possible to reuse the code—the general code applicable to all *ImageFiles* can be moved to that class. More specific code relating to *RasterFile* and *VectorFiles* can move to those classes, and concrete details on image formats such as JPEG or SVG can go into corresponding classes such as *JPEGFile*, *SVGFile* etc.

To summarise: with inheritance, we are exploiting commonalities between implementations by abstracting the interface. Now, code can be written in terms of the common base interface. When specific derived implementations need to be assigned to the base interface references, the user code need not change. Hence, this approach leads to re-usability and flexibility in design. However, this fundamental benefit of inheritance is broken if the derived types cannot be assigned to the base types. Liskov's Substitution Principle (LSP) describes such a situation, and hence is a cardinal rule to follow in OOP.

The informal definition of LSP is this: "Derived classes must be usable through the base class interface without the need for the user to know the difference." I know this description is difficult to understand, and I'll explain LSP using two examples from the Java library, to illustrate what it means to violate this principle.

Every computer science student who has taken a data structure course knows that a stack is not a vector. A vector is just like an array, only that it can grow in size. So we can insert or delete elements from anywhere in the vector. However, stack is a LIFO (Last In First Out) data structure: we can insert and remove only from one end of the data structure. Hence, a stack is not a vector; maybe a stack can be implemented using a vector. In JDK, these two container classes share an inheritance relationship: *Stack* extends *Vector*. For this reason, we can add